

DAY 16

Difference Array

Master Range Updates with $O(1)$ Boundary Operations

CORE IDEA

Store changes at boundaries.
Recover values using prefix sum.

COMPLEXITY

Q range updates $\rightarrow O(Q)$
Final reconstruction $\rightarrow O(N)$

GITHUB

ritesh2687/dsa_ppt_post
Day 16 resources
[Open GitHub \$\rightarrow\$](#)

Today's Learning Roadmap

Follow the pattern from intuition to interview readiness

01

What is it?

02

Why use it?

03

Boundary trick

04

C++

05

Dry run

06

Complexity

07

LeetCode

08

Interview

INTERACTIVE FLOW → pause at each step and predict the next array

NEXT

TOP

1. 🧠 What is a Difference Array?

A technique for handling many range updates efficiently

- A Difference Array stores the change between neighboring values.
- Instead of modifying every element in $[L, R]$, we mark where the change starts and where it stops.
- Prefix sum then propagates those changes and reconstructs the final array.
- Think: CHANGE $\rightarrow$ ACCUMULATE $\rightarrow$ FINAL ARRAY

```
diff[0] = arr[0]
diff[i] = arr[i] - arr[i - 1]    // i > 0

final[i] = diff[0] + diff[1] + ... + diff[i]
```

[NEXT](#)[TOP](#)

2. ? Why do we need it?

The problem: repeated range updates can be expensive

NAIVE UPDATE

For $[L, R] += X$:

```
for (int i = L; i <= R; i++)  
    arr[i] += X;
```

One update may touch many elements.
With Q large updates, work can approach $O(NQ)$.

DIFFERENCE ARRAY

For $[L, R] += X$:

```
diff[L] += X;  
diff[R + 1] -= X;
```

Only TWO boundary positions change.
Each update is $O(1)$.

[NEXT](#)[TOP](#)

3. The Boundary Trick

Why two operations are enough

Range update: [2, 6] += 5



```
diff[2] += 5;  
diff[7] -= 5;
```

NEXT

TOP

Boundary Update → Prefix Sum

See the +5 propagate, then cancel

Difference array after marking boundaries

0	0	5	0	0	0	0	-5
0	1	2	3	4	5	6	7

Prefix sum → final values

0	0	5	5	5	5	5	0
0	1	2	3	4	5	6	7

The +5 begins at index 2 and remains active through index 6.
At index 7, -5 cancels it.

[NEXT](#)[TOP](#)

The Core Formula

Memorize the pattern, not the code

RANGE UPDATE

$[L, R] += X$

$\text{diff}[L] += X$

$\text{diff}[R + 1] -= X$

WHY $R + 1$?

We want X to affect R ,
but NOT $R + 1$.

So cancellation happens at $R + 1$.

SAFE BOUNDARY

Use diff of size $N + 1$, then the $r+1$ position can safely hold the cancellation.

 **Classic bug: $\text{diff}[R] -= X \rightarrow$ the last element loses the update.**

NEXT

TOP



Worked Example

One range update from start to finish

Initial array

10	10	10	10	10
0	1	2	3	4

Update: [1, 3] += 5

```
diff[1] += 5  
diff[4] -= 5
```

```
Prefix sum reconstructs:  
[10, 15, 15, 15, 10] + base values  
→ [10, 15, 15, 15, 10]
```

[NEXT](#)[TOP](#)



C++ Implementation

Interview-ready implementation

```
vector<int> rangeAdd(vector<int> arr,  
                    vector<vector<int>>& updates) {  
    int n = arr.size();  
    vector<int> diff(n + 1, 0);  
  
    diff[0] = arr[0];  
    for (int i = 1; i < n; i++)  
        diff[i] = arr[i] - arr[i - 1];  
  
    for (auto &q : updates) {  
        int l = q[0], r = q[1], x = q[2];  
        diff[l] += x;  
        diff[r + 1] -= x;  
    }  
  
    int cur = 0;  
    for (int i = 0; i < n; i++) {  
        cur += diff[i];  
        arr[i] = cur;  
    }  
    return arr;  
}
```

[NEXT](#)[TOP](#)



Complete Dry Run

Multiple updates

- Initial array: [10, 10, 10, 10, 10]
- Update 1: [1, 3] += 5
- Update 2: [2, 4] += 2
- After boundary marking, only a few diff positions change.

Difference changes:

[10, 5, 2, 0, -5]

Prefix sum:

10 → 15 → 17 → 17 → 12

FINAL:

[10, 15, 17, 17, 12]

[NEXT](#)[TOP](#)

⚡ Three Range Updates

Practice seeing boundaries instead of individual elements

n = 8

UPDATE 1

$[1, 4] += 3$

UPDATE 2

$[2, 6] += 5$

UPDATE 3

$[0, 3] += 2$

Boundary marks combine into:

diff $\rightarrow [2, 3, 5, 0, -3, 0, 0, -5]$

Prefix sum $\rightarrow [2, 5, 10, 10, 7, 7, 7, 2]$

NEXT

TOP



Time & Space Complexity

Why this technique scales

Build diff

$O(N)$ • one pass

Each update

$O(1)$ • two boundary writes

Q updates

$O(Q)$ • no range loop

Reconstruct

$O(N)$ • prefix sum

TOTAL

$O(N + Q)$ time • $O(N)$ extra space

NEXT

TOP

LeetCode Practice

Three problems to lock in the pattern

1109 • Corporate Flight Bookings

Difficulty: Medium

Pattern:

Range increments + prefix sum.

Hint:

Mark first flight +seats and last+1 - seats.

1589 • Maximum Sum Obtained of Any Permutation

Difficulty: Medium

Pattern:

Count how often each index is requested.

Hint:

Build frequency with boundary updates.

1854 • Maximum Population Year

Difficulty: Easy

Pattern:

Birth = +1, death = -1.

Hint:

Prefix sum events and find the maximum.

[NEXT](#)[TOP](#)



Pattern Recognition

Know when Difference Array is the right tool

THINK DIFFERENCE ARRAY WHEN...

- Many range updates
- Updates are known beforehand
- Final array/result is needed
- Operation looks like $[L,R] += X$
- You can process updates offline

DON'T CONFUSE IT WITH...

Prefix Sum $\rightarrow$ fast static range sums

Fenwick Tree $\rightarrow$ online point update + queries

Segment Tree $\rightarrow$ flexible range updates/queries

Key question:

“Offline or online?”

[NEXT](#)[TOP](#)



Common Mistakes

Avoid these interview and coding traps

- 1. Using `diff[R] -= X` instead of `diff[R+1] -= X`.
- 2. Forgetting the extra `N+1` slot for safe cancellation.
- 3. Mixing 0-indexed and 1-indexed problem statements.
- 4. Using `int` when accumulated values require `long long`.
- 5. Using Difference Array for online query/update workloads where a data structure is required.

RULE OF THUMB: if you need answers while updates are still arriving, think beyond Difference Array.

[NEXT](#)[TOP](#)



Interview Tip

One question can reveal the intended data structure

**“Do I need the result only after all updates,
or do I need to answer queries while updates are happening?”**

OFFLINE

All updates first

↓

Difference Array
+ Prefix Sum

ONLINE

Updates + queries mixed

↓

Fenwick Tree / Segment Tree

INTERVIEW MOVE

Clarify constraints
before coding.

NEXT

TOP



Mini Challenge

Pause here and solve before revealing the answer

n = 7

Updates:

$[1, 4] += 4$

$[2, 5] += 3$

$[0, 2] += 2$

YOUR TASK

Build the difference array using only boundary updates.
Then take a prefix sum.

ANSWER

$[2, 6, 9, 7, 7, 3, 0]$

Interactive cue: cover the ANSWER card while solving.

NEXT

TOP



Day 16 Complete

Difference Array = store changes, then accumulate

RANGE UPDATE

$[L, R] += X$



BOUNDARIES

$\text{diff}[L] += X$
 $\text{diff}[R+1] -= X$



PREFIX SUM

FINAL ARRAY

KEY TAKEAWAYS

- Each range update becomes $O(1)$
- Reconstruction costs $O(N)$
- Total for Q updates: $O(N + Q)$
- Watch $r+1$ and indexing carefully



TOMORROW — DAY 17

Range Queries & Fenwick Tree Basics
When updates and queries happen online.



GITHUB

https://github.com/ritesh2687/dsa_ppt_post

TOP